

Simulador de AWS S3 sobre sockets

Principios de Sistemas Operativos - Tercer proyecto

Introduccion

Este proyecto implementa un servidor y un cliente en lenguaje C que simulan operaciones basicas de AWS S3. El cliente aws-s3 envia comandos al servidor aws-s3_server mediante sockets TCP. El servidor almacena cada bucket remoto como un unico archivo local.

La implementacion soporta objetos binarios porque el protocolo separa las lineas de control del contenido de archivos, enviando siempre el tamano exacto del payload antes de transmitir los bytes.

Descripcion del problema

El sistema debe permitir crear, listar, copiar, mover, eliminar y sincronizar objetos entre rutas locales y rutas S3 simuladas. Las rutas S3 usan el formato s3://bucket/prefijo/objeto.

S3 no maneja directorios reales dentro del bucket. Las carpetas son prefijos dentro de la clave completa del objeto. Por eso el servidor almacena claves planas como dir/subdir/archivo.txt y el cliente las presenta como carpetas simuladas al listar.

Comandos implementados

Comando	Funcion
ls	Lista buckets o contenido de un bucket/prefijo.
mb	Crea un bucket.
cp	Copia local->S3, S3->local o S3->S3. Soporta --recursive.
mv	Mueve mediante copia y eliminacion del origen.
rm	Elimina objetos o prefijos con --recursive.
sync	Sincroniza local->S3 o S3->local. Usa --delete para eliminar sobrantes.
rb	Elimina buckets vacios o con contenido usando --force.

Definicion de estructuras de datos utilizadas

bucket_header_t: bloque fijo al inicio de cada archivo bucket. Contiene magic, contadores, tabla de objetos y tabla de espacios libres.

bucket_entry_t: representa un objeto almacenado. Guarda bandera used, key, offset y size. La key es el path plano completo dentro del bucket.

bucket_hole_t: representa un espacio libre dentro del archivo bucket. Guarda bandera used, offset y size. Se genera al borrar un objeto o reemplazarlo por otro de distinto tamano.

s3_uri_t: estructura usada por el cliente para separar una URI s3:// en bucket y key.

Componentes principales del programa

src/common.c contiene funciones compartidas para sockets, lectura y escritura exacta de bytes, lectura de lineas, manejo de rutas y creacion de directorios.

src/bucket.c contiene la administracion fisica de buckets. Implementa crear, borrar, listar, guardar, recuperar, copiar y eliminar objetos. Tambien administra la lista de espacios libres con primer ajuste.

src/server.c abre un socket TCP, acepta conexiones y atiende cada cliente en un proceso hijo mediante fork. Cada conexion procesa un comando.

src/client.c implementa la interfaz de linea de comandos aws-s3, parsea rutas locales y S3, envia comandos al servidor y transmite o recibe payloads binarios.

Mecanismo de creacion de archivos y comunicacion

Cada bucket se crea como buckets/.bucket. Al crearlo se escribe un header fijo con magic S3BKT01 y tablas vacias. El contenido de objetos se escribe despues del bloque de directorio.

El protocolo es textual para los comandos y binario para los archivos. Por ejemplo, PUT envia la linea PUT bucket key size seguida exactamente por size bytes. GET responde OK size seguido por los bytes del objeto.

Cuando se sube un objeto ya existente con el mismo tamano, se sobrescribe en su offset actual. Si el tamano cambia, el espacio anterior se agrega a la lista libre y el nuevo contenido se ubica usando primer ajuste o al final del archivo.

Pruebas de rendimiento y funcionamiento

Se incluyo una prueba automatizada en tests/smoke.sh. La prueba compila, levanta un servidor local, ejecuta operaciones principales y compara los archivos recuperados con cmp.

```
make test
./aws-s3 mb s3://demo
./aws-s3 cp tmp-smoke/local/a.txt s3://demo/
./aws-s3 cp tmp-smoke/local s3://demo/copia/ --recursive
./aws-s3 cp s3://demo/a.txt tmp-smoke/out/a.txt
cmp tmp-smoke/local/a.txt tmp-smoke/out/a.txt
./aws-s3 cp s3://demo/copia/ tmp-smoke/out/copia --recursive
./aws-s3 cp s3://demo/copia/ s3://demo/copia2/ --recursive
./aws-s3 mv s3://demo/copia2/ s3://demo/copia3/ --recursive
./aws-s3 rm s3://demo/copia/ --recursive
./aws-s3 sync tmp-smoke/local s3://demo/sync/ --delete
./aws-s3 sync s3://demo/sync/ tmp-smoke/out/sync
./aws-s3 rb s3://demo --force
```

Resultado observado: la prueba smoke finaliza con 'Pruebas smoke completadas'. Las comparaciones cmp verifican que los bytes recuperados coinciden con los archivos originales.

Para archivos nuevos o modificados, sync compara la existencia y el tamaño. Esta decisión mantiene el formato del bucket simple y suficiente para demostrar la sincronización solicitada.

Conclusiones

La implementación demuestra un sistema de almacenamiento remoto basado en objetos planos y prefijos simulados, con comunicación cliente-servidor por sockets y soporte para archivos binarios.

El uso de un bloque de directorio fijo simplifica la recuperación de metadatos y permite que cada bucket exista como un solo archivo. La lista de espacios libres permite reutilizar regiones liberadas sin compactar el archivo completo.

Las pruebas automatizadas cubren creación de buckets, carga, descarga, copia recursiva, movimiento, eliminación recursiva, sincronización y eliminación forzada de buckets.